



# Chronos.

**AI-Powered Legacy Code Modernization with IBM Bob**

**Team bubududu**

IBM Bob 2.0 Hackathon — Sept 2026



# Legacy C/COBOL/Fortran code nobody dares to touch.



## It still runs the money and the science

Financial and scientific computing sits on decades-old C that is in production right now.



## Manual security audits don't scale

A human reviewer reads a few hundred lines an hour. The backlog is millions of lines deep.



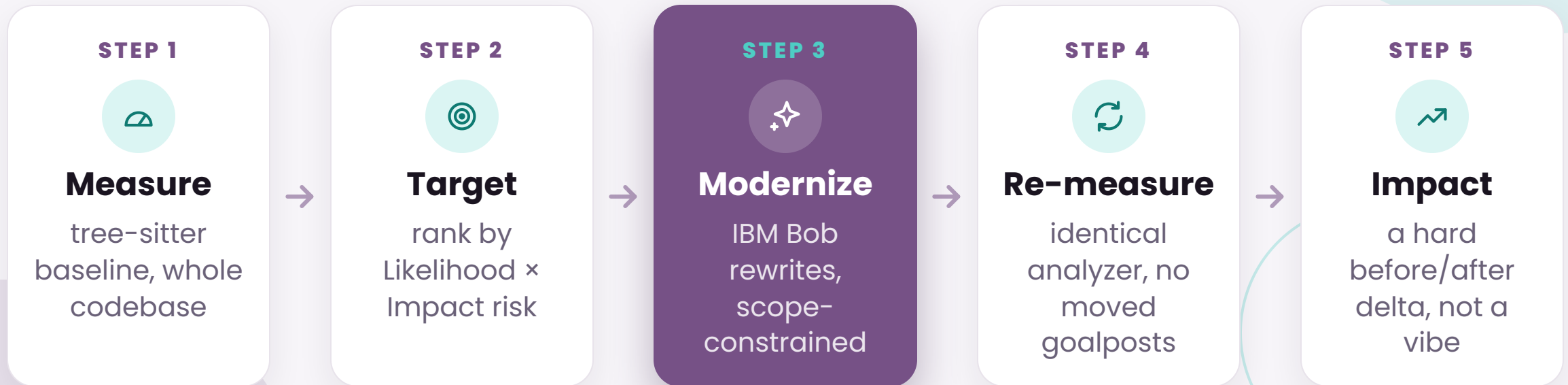
## "Modernization paralysis"

Fear of breaking working logic blocks every fix. Unsafe code ships, year after year.

## 02 OUR APPROACH

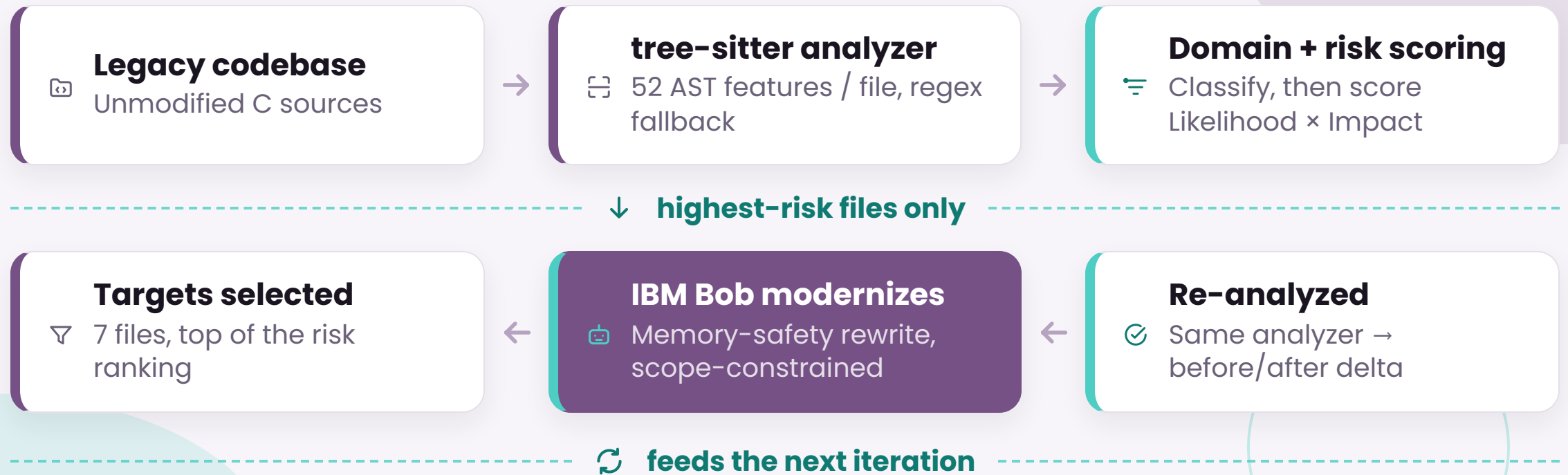
# A closed measurement loop.

Prove the improvement with the same analyzer — before and after.



🔄 The loop repeats until the risk metrics stop moving — measurement is the contract.

# One pipeline, six stages.



# Autonomous, but on a short leash.

Report on all 7 sites

Sites already guarded (no chan

| Site | Line | Variable |
|------|------|----------|
| 1    | ~213 | res      |
| 2    | ~232 | slug     |
| 3    | ~293 | repo     |
| 4    | ~841 | slug     |
| 5    | ~867 | slug     |

Skills 582

Messages 15.6k

Estimated breakdown

Reserved for model response 64.0k

Available space 177.6k

Already guarded — if (res) { memset/sprintf... } wraps all uses. If malloc fails, NULL is returned to caller. Correct pattern for a builder function.

Already guarded — same if (slug) wrapping pattern. Returns NULL on failure.

Already guarded — same if (repo) wrapping pattern. Returns NULL on failure.

Already guarded — same if (slug) wrapping pattern in clib\_package\_url(). Returns NULL on failure.

Already guarded — same if (slug) wrapping pattern in clib\_package\_url\_from\_repo(). Returns NULL on failure.

@demo/samples/after/clib/clib-packa... just now 0.532

@demo/samples/after/clib/clib-packa... 43 min ago 0.904

@chronos-legacy-dataset/c\_financial\_... 6 hrs ago 1.04

@demo/samples/before/kopen/kope... 6 hrs ago 0.313

@demo/samples/before/khmm/khm... 6 hrs ago 0.548

Recent(1)

/init 7 hrs ago 3.46

IBM Bob autonomously modernizes legacy C for memory safety — **7 files**, checking every allocation site, guided by strict scope constraints.

- ✔ Touch only the listed files.
- ✔ Never the analyzer.
- ✔ Never the measurement baseline.

## 05 BUG CATCH #1

# Bob found a crash bug we didn't ask about.

clib-package.c — clib\_package\_new\_from\_slug\_with\_package\_name()

### ❗ BEFORE

```
res = http_get(json_url);  
json = res->data;  
_debug("status: %d", res->status);  
if (!res || !res->ok) {  
    goto download;  
}
```

### ✅ AFTER

```
res = http_get(json_url);  
if (!res || !res->ok) {  
    goto download;  
}  
json = res->data;  
_debug("status: %d", res->status);
```

⚠️ **A guaranteed crash on any network failure.** `http_get()` returns NULL on failure, and the original code dereferenced it one line *before* its own NULL check.

# A heap overflow, self-detected.

kopen.c — http\_open(), reading the response header into a 64 KB calloc buffer

## ❗ BEFORE

```
while (read(fd, buf + l, 1)) {  
    if (buf[l] == '\n' && l >= 3)  
        if (strcmp(...) == 0) break;  
    ++l;    /* no upper bound */  
}  
buf[l] = 0; /* l reaches 0x10000 */
```

## ✅ AFTER

```
while (read(fd, buf + l, 1)) {  
    if (buf[l] == '\n' && l >= 3)  
        if (strcmp(...) == 0) break;  
    ++l;  
    if (l >= 0xffff) break; /* bound */  
}  
buf[l] = 0; /* always in range */
```

⚠️ A header larger than 64 KB walks `l` to `0x10000`, and `buf[l] = 0` writes **one byte past the heap allocation**. The loop now stops one byte short.



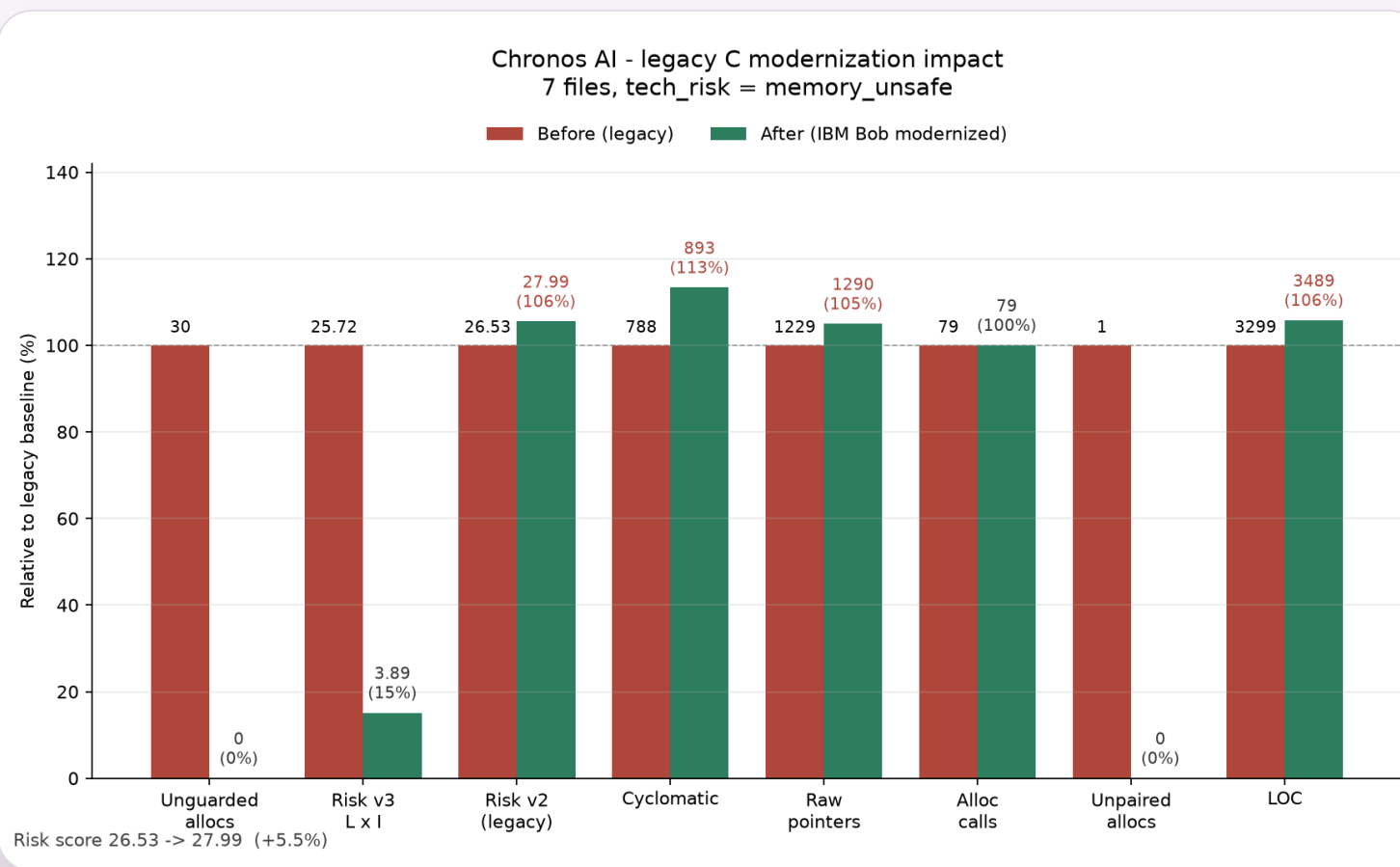
30 → 0

Unguarded allocation sites, **100% eliminated**  
across 7 files.

MEASURED BY THE SAME ANALYZER, BEFORE AND AFTER



# Every metric, measured twice.



Unguarded allocs **30 → 0**

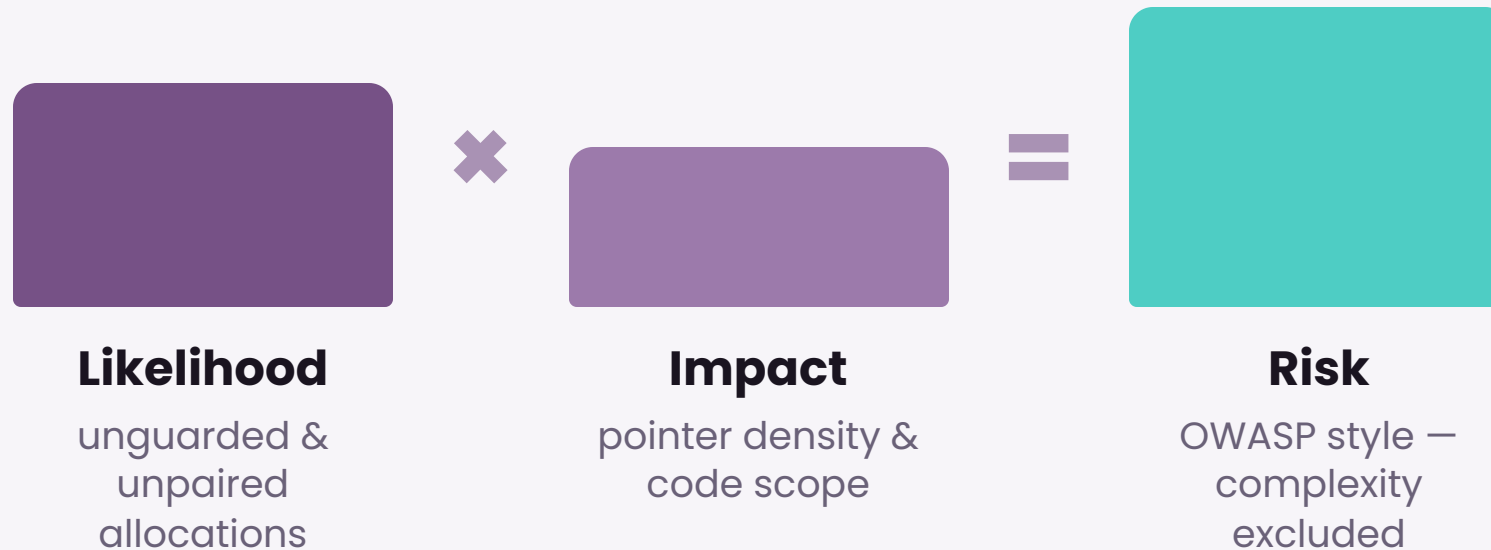
Unpaired allocs **1 → 0**

Risk (L × I) **-84.9%**

Both allocation metrics **-100%**.  
Risk collapsed **25.72 → 3.89** — a  
near-zero residual — for only  
**+5.8%** more code.

# Why we didn't just use one risk number.

Adding defensive NULL-checks **raises** cyclomatic complexity. A blended score would punish the fix.



**Shin & Williams (2010)** found only a weak correlation ( $r \approx 0.30$ ) between cyclomatic complexity and real vulnerabilities — a poor proxy for safety, and a terrible one for measuring a safety fix.

# Who needs this, and what's next.

## Who benefits

-  **Banks & fintech** running settlement, pricing and risk engines on C that predates their compliance regime.
-  **Scientific-computing teams** maintaining numerical kernels and bioinformatics tooling nobody rewrote.
-  Both get an **auditable paper trail**: a measured delta, not a claim.

## Next steps

-  Close the remaining **clib-package.c** gap — the one file whose risk score stayed flat.
-  Scale from 7 files to the **full mined legacy dataset**.
-  Ship it as a **CI gate**: block any merge that raises the Likelihood × Impact score.

# Thank you.

**Team bubududu**

 [github.com/Phong-uet/chronos-ai](https://github.com/Phong-uet/chronos-ai)

 Built with IBM Bob